

Beyond Prompts: 开源 LLM Agent Harness 中指令合规控制的文献与代码证据

核验日期: 2026-08-20

材料范围: 《相关工作.pptx》23 页、11 篇论文/资料、16 个本地 Git 仓库

本地证据: [论文索引](#) · [PDF 完整性清单](#) · [仓库提交清单](#) · [代码证据矩阵](#) · [测试记录](#)

摘要

本报告围绕《相关工作.pptx》提出的核心问题展开：开源、可部署的 LLM agent harness 依靠哪些代码级机制来规范、限制、检测、恢复、授权和验证指令执行。对 11 篇相关论文和 16 个本地仓库的核验表明，现有工作已分别覆盖指令可验证性、结构化输出、工具协议、人工审批、运行时隔离、后置条件和测试实践，但不同论文中的 “guardrail” “constraint” “verification” 经常指向不同生命周期位置与不同保证强度。最稳健的经验结论不是 “某种单一机制解决指令遵循”，而是代码所有权和执行位置决定规则能否真正改变行为：prompt 中的规则仍可能被模型忽略；输出 evaluator 能检测但通常不能阻止副作用；工具审批和内核策略可在执行前阻断；恢复逻辑则只在失败已被检测后发挥作用。为避免把这些机制错误地合并计数，本报告建议以具体 enforcement point 为分析单元，采用证据等级、生命周期位置、失败语义与绕过面共同编码，并将 PPT 中的初始分类计数保留为待独立复核的探索性结果。

1. 研究问题与边界

PPT 的目标可整理为一个主问题：在开源 LLM agent 系统中，哪些代码级控制被用于调节、约束、检测、恢复、授权或验证指令执行；这些控制如何分布于项目、执行生命周期和保证类型中；多个控制又如何组合。该问题比传统 instruction-following benchmark 更接近软件工程中的 enforcement：不仅问模型答得是否符合要求，还问系统在工具调用、状态变更和失败恢复时做了什么。

本报告纳入能够观察到可执行控制点的开源框架、guardrail、benchmark 和研究 artifact。只在 system prompt/README 中提出建议而没有实现接线的内容，记作 specification 或 E0，不记作强制控制；与指令合规无直接关系的通用 RAG、记忆和性能优化不计入。benchmark 中的 checker、reward 和 postcondition 被保留，但明确标为执行后检测，不与执行前阻断混写。

从指令到副作用：Agent Harness 控制链

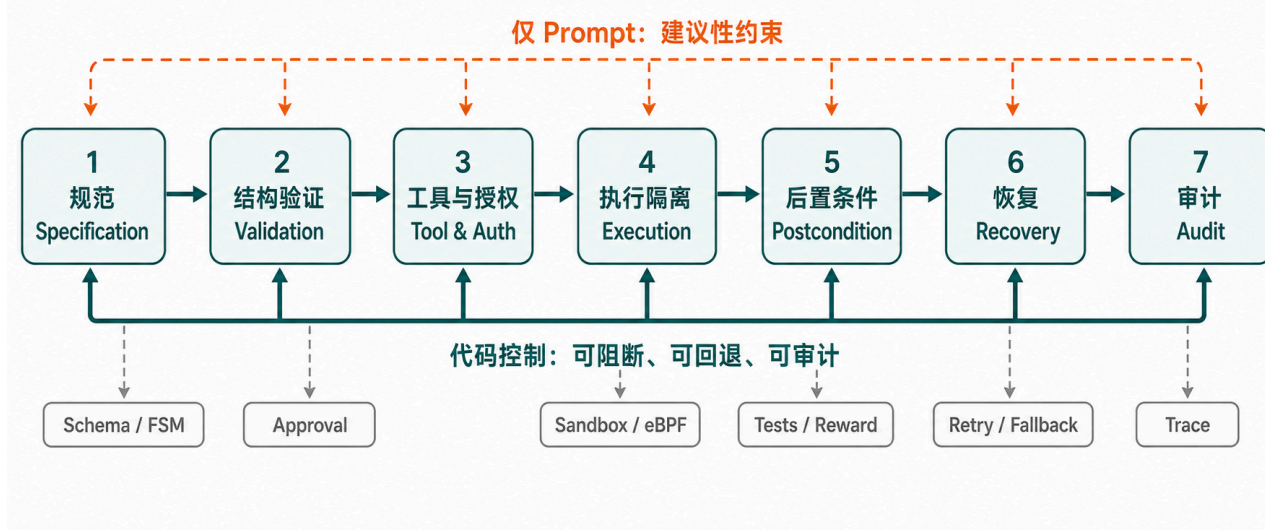


图 1 从规范、结构验证、工具授权、执行隔离到后置条件、恢复和审计的控制链。虚线表示仅依赖提示的建议路径，实线表示代码所有的控制路径。该图是概念综合，不是论文原图。

2. 方法与证据规则

本轮采用叙述性文献综述与仓库取证相结合的方法。首先从 PPT 的文本、备注和超链接提取候选论文与术语，再通过论文首页、arXiv/DOI、正式会议或期刊页面、官方项目页核验题名、作者、年份、发表状态、样本量和仓库。随后将可公开获取的 11 份 PDF 下载到本地，验证文件头、页数、文本抽取和 SHA-256。对存在代码的工作，固定到本地 Git commit，围绕策略判定、schema 验证、工具分发、人工审批、运行时阻断、后置条件和恢复路径搜索实现与测试。

证据等级定义为：E0 仅有文档、提示或论文声称；E1 能定位实现路径；E2 有针对该控制的自动化测试；E3 有对照、消融或行为测量；E4 有独立复现或生产证据。等级表示“本轮找到的证据强度”，不是项目总质量评分。对一个机制还需记录它在生命周期中的位置、是否能阻断副作用、异常时 fail-open 还是 fail-closed、是否保留跨步状态、调用方是否可绕过，以及它与其他控制的组合顺序。

本地复现保持克隆仓库不变，也未安装额外依赖。Enterprise harness 的 guardrail 子集 23/23 通过，但全量测试为 25/34 通过；IFEval 与 JSONSchemaBench 的运行测试分别被缺少 `absl` 与 `dacite` 阻断。这些结果被视为环境与快照层面的复现边界，而不是选择性忽略。

3. 文献综合

3.1 从“模型是否遵守”到“约束是否可执行”

IFEval 和 AGENTIF 代表指令遵循的可验证评测路线。IFEval 用可编程 checker 对 25 类指令进行严格与宽松判定，发行数据包含 541 个实例；AGENTIF 将场景扩展到 50 个 agent 应用、707 条长指令和平均 11.9 个约束，并以 constraint success rate 与 instruction success rate 聚合结果。二者揭示了长上下文、条件约束和多约束组合对模型的困难，但它们主要回答“结果是否合规”，并不直接提供工具调用之前的授权或隔离。

这一区别在代码中非常清楚。IFEval 的 strict evaluator 对各 checker 结果使用 `all()` 聚合，loose evaluator 通过文本变体提供上界；AGENTIF 动态加载 checker 并聚合 CSR/ISR。它们是可执行 oracle，但如果 agent 已经发送邮件、删除文件或修改数据库，执行后的低分并不能撤销副作用。因此，benchmark 证据应编码为 post-hoc verification，而不是 prevention。

3.2 结构化输出解决语法边界，但不自动解决语义与价值约束

JSONSchemaBench 以 9,558 个 schema 比较 Guidance、Outlines、Llama.cpp、XGrammar、OpenAI 和 Gemini 等结构化输出路径，关注效率、覆盖率与质量。其代码使用 JSON Schema Draft 2020-12 同时验证 schema 与实例，并区分框架声称覆盖与经验覆盖。2026 年的软件工程结构化输出研究进一步指出，语法或 grammar 约束能够抑制一类解析错误，却不能消除结构关系错误和 value errors。

这意味着 schema 是必要但局部的控制。它能确保 `amount` 字段存在且类型正确，却不能单独判断转账是否经用户授权、对象是否是正确账户、操作是否符合业务状态。将“可解析”写成“指令合规”会夸大保证。更合理的组合是 schema 负责形状，semantic validator 负责上下文，authorization gate 负责能力，postcondition 负责最终状态。

3.3 Prompt template 是规范层，不是强制层

From Prompts to Templates 对开源 LLM 应用中的 prompt template 进行组件化分析，展示角色、上下文、任务、约束、示例和输出格式等规范如何组合。这为 specification taxonomy 提供经验基础，但不能据此推断约束在运行时得到执行。 τ -bench 更直接地呈现这种边界：零售环境规则明确写着先认证、数据库变更前取得用户授权、一次只调用一个工具，但这些内容首先是传给模型的规则文本；环境真正执行的代码主要负责工具分发、错误返回和终止后的数据库/输出比较。

因此，PPT 中的“Specification”应作为独立类别保留，同时设置一个 `enforcement_owner` 字段。规则由 prompt/model 所有时，保证依赖模型服从；规则由 runtime/tool wrapper/OS 所有时，系统能在模型失误时仍然拒绝操作。二者可以使用同一自然语言规则，却不具有同等保证。

3.4 Harness 将控制分布到工具边界、审批、恢复和审计

OpenAI Agents Python、NVIDIA NeMo Guardrails、Microsoft Agent Governance Toolkit 与 Purple Llama 展示了典型的中间层控制。OpenAI Agents 将 strict schema、工具输入/输出 guardrail、`needs_approval`、暂停和恢复接入工具执行；测试表明 run 可在审批处暂停，批准后恢复且执行一次。代码同时明确记录一个重要绕过面：直接包装 callable 的路径可能绕过 schema、guardrail、timeout 与 tracing。NeMo 在输入、输出、tool call 和 tool result 上设置 rails，并校验工具名与调用 ID；普通 rail 异常会产生不安全判定，但上游 HTTP 错误会继续抛出，故不能把它简化为“任何失败都阻断”。

Governance Toolkit 将策略求值、verdict 正规化、人工审批和审计串成路径；其端到端测试验证批准时工具只执行一次、拒绝时不执行。Purple Llama 的 LlamaFirewall 将扫描结果扩展为 `ALLOW`、`BLOCK`、`HITL` 等决定，CodeShield 可以按严重程度返回 block/warn；然而扫描器只有被调用方接入实际执行路径时才会成为强制控制。共同结论是：机制名称不够，必须检查调用图中 verdict 是否最终支配副作用。

3.5 系统级隔离提供更强的副作用保证，但适用面更窄

ActPlane 将控制移到 Linux/eBPF/LSM 层。其策略可以对 agent 来源的执行、文件写入、网络连接和破坏性命令采取 block/kill；BPF 实现会在规则命中时返回 `-EPERM` 或发送 `SIGKILL`。这是本样本中最清晰的“模型无论是否服从，副作用边界仍可阻断”的代码证据。论文报告 1.9%–8.4% 开销，但该结果来自特定 Linux/eBPF 原型，不能直接外推到跨平台、SaaS API 或远程浏览器工具。

它也说明执行隔离不能替代上层语义：内核可以阻止某个路径的文件写，却不知道一封措辞正确的邮件是否发送给了错误客户。完整 harness 因而需要分层组合，而不是选择唯一 guardrail。

3.6 测试与生产证据仍然不足

Testing Practices in Open-Source AI Agent Frameworks and Applications 调查 39 个框架和 439 个应用，并手工分析 759 个测试函数，发现工具和 workflow 组件受到较多关注，而特定于 agent 非确定性或专门 evaluator 的测试比例很低。Tangent 从测试生成角度报告 2,572 个方法、240 个模块、23 个模式和 10 次访谈，但截至核验日公开 artifact 仓库为空，本轮无法把论文结果提升为代码级可复现证据。

Measuring Agents in Production 汇集 20 个访谈/案例和 86 个已部署或试点系统，强调人类评估、步骤数限制、规则评估与模型评估的组合。PPT 中出现的 306 是未筛选受访者而不是最终分析样本。该研究支持“生产评估是组合式的”，但没有公开仓库可供本轮追踪到 enforcement point。

From Prompts to Contracts 提供了较少见的消融式证据：在限定企业案例的 270 次组合实验中，论文报告 code-owned harness utility 为 120/120，外部 guardrail 为 88/120；仓库代码显示 prompt-only 分支可以记录 violation 却仍返回原始输出，而内置 harness 在验证失败时进入确定性 fallback。本地 guardrail 子集测试通过，但全量测试仍有 9 个失败，因此本报告把它列为 E3 的研究证据，而不是已独立完整复现的 E4。

4. 代码取证的主要发现

发现	直接代码证据	推论边界
规则能否改变行为取决于 enforcement owner	<code>τ-bench</code> 规则文本 vs. ActPlane <code>-EPERM/SIGKILL</code> ；Enterprise prompt-only vs. code-owned 分支	不能从 README 或 system prompt 推断强制性。
工具调用是最常见的可执行控制点	OpenAI Agents guardrail/approval；NeMo tool call/result rails；Governance Toolkit verdict	只覆盖通过框架包装的工具；直接 callable、外部 API 或旁路可能绕过。
输出结构与语义/授权是不同层	JSONSchemaBench schema validator；业务规则与 postcondition	JSON 合法不等于操作正确或获授权。
Postcondition 强于无检测，但弱于事前阻断	<code>τ-bench</code> 数据库哈希/输出项奖励；AgentDojo task utility/security checks	已产生的不可逆副作用可能无法恢复。
Fail-closed 需要逐异常路径核验	NeMo 普通 rail 错误阻断、HTTP 错误抛出；verdict 正规化测试	“有 guardrail”不自动说明异常语义。
恢复机制依赖可靠检测	Enterprise deterministic composer fallback；审批暂停/恢复	检测漏报或状态恢复不完整时，recovery 不生效。
公开 artifact 并不等于可复现	Tangent 空仓库；IFEval Windows 特殊物化；部分缺依赖	必须记录 commit、依赖、测试结果与例外。

完整的逐仓库证据、文件与行号见 [代码级证据矩阵](#)。本轮最值得进一步做深的六个核心项目是 Agent Governance Toolkit、AgentDojo、NeMo Guardrails、OpenAI Agents Python、Purple Llama 与 τ^2 -bench；ActPlane 和 Enterprise harness 适合作为更强 enforcement/消融对照。AGENTIF、IFEval、JSONSchemaBench 作为 evaluator 层辅助样本，不应与 runtime harness 混为同一总体。

5. 对 PPT 的核验与修正

PPT 内容	核验结果	建议写法
AGENTIF 平均长度/约束	论文摘要与数据支持平均 1723 words、11.9 constraints；结论另写 1717 tokens	引用具体章节并保留单位差异，避免合并成一个数字。
IFEval “约 500”	概括可接受，发行实例为 541	报告中写 “541 个实例、25 类指令”。
JSONSchemaBench “10K”	精确为 9,558 schemas	表格用 9,558，正文可写约 9.6K。
Structured Output Control 的 RQ2/RQ3	PPT 顺序与论文相反	RQ2 为 case study；RQ3 为 grammar/regex/TTMG 评估。
Testing practices 的 “~1%”	只对应特定 agent 测试，不是所有测试	明确分母和被编码类别。
Production agents 的 306	是未筛选受访者；最终分析为 20 个访谈/案例与 86 个系统	不把 306 写成生产系统样本量。
Harness survey 的 “170+” 与分类计数	来自不同项目快照/分母	固定访问日期与 commit，分别报告，不相加。
机制分布 10/7/7/6...	无法从 PPT 单独恢复项目清单、编码表和去重规则	标为 provisional，待按仓库×enforcement-point 重新编码后再发表。

PPT 最后一页的机制计数适合作为探索性观察，但当前不能作为可复核结论。尤其 “结构验证 10” “输出约束 7” 等数字混合了框架、benchmark 和研究 artifact 的可能性较高，而同一项目又可能贡献多个机制。后续应公开采样框、项目版本、编码单元、双人编码一致性和每个计数对应的代码证据。

6. 建议的实证研究设计

研究对象应分层抽样，而不是把所有含 “agent” 字样的仓库合并。第一层是 deployable runtime/harness，第二层是 guardrail/治理中间件，第三层是 evaluator/benchmark，第四层是研究 prototype。核心结论主要从前两层产生，后两层用于观察验证方法 and 研究可复现性。对已经声明 deprecated/outdated 的 τ -bench，应以 τ^2 -bench 作为现代复现主对象，并保留旧版用于历史对比。

编码单元建议定义为 “一个能够定位到调用图的 enforcement point”，而非 “一个仓库有/无某类机制”。每条记录至少包含项目、commit、入口、被保护对象、生命周期位置、机制类型、enforcement owner、fail-open/closed、状态持久性、bypass surface、测试证据和证据等级。两个研究者独立编码后，可用 Cohen’s kappa 或 Krippendorff’s alpha 报告一致性；冲突通过共同追踪调用路径解决，而非仅凭术语表投票。

建议回答三个可检验的子问题。第一，控制在规范、执行前、执行中、执行后、恢复和审计各阶段如何分布；第二，哪些控制真正支配副作用，哪些只提供检测或评分；第三，组合控制的顺序、失败语义和旁路如何影响保证。实验上可构造一组相同任务，在 prompt-only、schema-only、tool gate、postcondition、OS isolation 和组合条件下测量违规通过率、误阻断率、恢复成功率、延迟与 token/系统开销。这样可以把 “机制存在” 推进到 “机制在什么威胁模型下有效”。

7. 局限性

本报告是一次定向证据审计，不是完整系统综述。论文集合由 PPT 及其相邻研究扩展而来，可能遗漏未公开或非英文工作；仓库只固定到 2026-08-20 的可访问状态，未来提交可能改变实现。多数结论来自静态调用路径与仓库自带测试，只有少数论文提供消融或生产资料。Enterprise harness 的全量测试未完全通过，IFEval 与 JSONSchemaBench 因未安装依赖而未运行；Tangent artifact 为空。Windows 文件名限制使 google-research 的 IFEval 采用稀疏/归档物化，故其工作树不为 clean。以上限制都保留在清单中，避免把“已下载”“有仓库”和“已独立复现”混为一谈。

8. 结论

现有文献和代码共同支持一个分层观点：instruction compliance 不是模型层的单一能力，也不是增加一个 guardrail 即可解决的问题。规范层说明应该做什么，结构层保证输出形状，工具与授权层限制能力，执行隔离层阻断副作用，后置条件检测结果，恢复层处理失败，审计层保存责任链。真正决定保证强度的是每个控制的执行位置、所有权和旁路，而不是其名称。

因此，后续研究应从“项目是否包含 guardrail”转向“具体控制点是否不可绕过、发生在何时、失败时怎样、由什么测试支持”。PPT 已经给出了合适的问题框架；本轮核验补上的关键部分，是把每个类别落到 commit、文件、行号、测试与证据等级，并把尚未可复核的计数明确降格为 provisional。

参考资料与本地附件

论文的题名、作者、发表状态、DOI/arXiv、官方代码及逐项校正见 [论文与资料索引](#)。11 份本地 PDF 与 SHA-256 见 [PDF manifest](#)，16 个仓库的 upstream、commit、分支和 clean 状态见 [repository manifest](#)。PPT 的 OOXML 文本抽取见 [related_work_pptx.md](#)，原始结构化结果见 [related_work_pptx.json](#)。

核心官方入口包括 [Agent Harness Engineering 项目页](#)、[awesome-agent-harness 目录](#)、[AGENTIF](#)、[JSONSchemaBench](#)、[ActPlane](#)、[Enterprise LLM Agent Harness](#)、[OpenAI Agents Python](#)、[NeMo Guardrails](#)、[Microsoft Agent Governance Toolkit](#) 与 [Purple Llama](#)。